

Introduction to C Programming Language

The **C language** was invented at Bell Labs by Dennis Ritchie in 1972 to allow the writing of the UNIX operating system.

The UNIX operating system, born in the late 1960s - early 1970s, was written directly in assembly language for the machines which it was intended. born in the late 1960s - early 1970s, was written directly in assembly language for the machines which it was intended. If the assembly language for writing such a system, it was nevertheless not easy to use. Such language is in fact a particular type of processor, which makes the whole system should be rewritten to make it work on a new architecture. This is its main creator, Ken Thompson, wanted to use a language more quickly evolved to rewrite UNIX.

Among the languages available at the time, BCPL (Basic Combined Programming Language for, which is a simplification of CPL), created by Martin Richards in 1966, was interesting. Without going into detailed descriptions, BCPL is a simple language, procedural, and not typed. Its simplicity makes it easy to create BCPL compilers on machines of the time, when resources were very limited (the first computer used by Keith Thompson was to launch a Unix PDP-7, which had a memory of 4000 words 18 bits, or less than 9 KB). Ken Thompson has evolved to design the B language, which he implemented the first UNIX machines. However, some limitations of language B were UNIX could be rewritten in this language.

From 1971, Dennis Ritchie B did evolve to address these issues. Like the programmers increment versions of their programs, Ritchie "Nudge" the letter B to call the new language C. This evolution is "stabilized" to 1973, from which UNIX and UNIX system utilities have been rewritten successfully in C.

Subsequently in 1978, Brian W. Kernighan documentation very active language, and finally publish the book with reference Ritchie The **C Programming Language**. Often called K & R C language as specified in the first edition of this book.

In the years that followed, the C language was carried on many other machines. These ports were often made at the beginning, from the pcc compiler Steve Johnson, but then the original compilers were developed independently. During these years, every C compiler was written in accordance with the specifications of K & R, but some added extensions, such as data types or additional features, or different interpretations of certain parts of the book (not necessarily very precise). Because of this, it was less easy to write **C programs** that can run unchanged on many architectures.

C is a general-purpose high level language that was originally developed by Dennis Ritchie for the Unix operating system. It was first implemented on the Digital Equipment Corporation PDP-11 computer in 1972.

The Unix operating system and virtually all Unix applications are written in the C language. C has now become a widely used professional language for various reasons.

- Easy to learn
- Structured language
- It produces efficient programs.
- It can handle low-level activities.
- It can be compiled on a variety of computers.

In 1972 Dennis Ritchie at Bell Labs writes C and in 1978 the publication of The C Programming Language by Kernighan & Ritchie caused a revolution in the computing world

- **In 1983, the American National Standards Institute (ANSI) established a committee to provide a modern, comprehensive definition of C. The resulting definition, the ANSI standard, or "ANSI C", was completed late 1988.**

C is a structured programming language, which means that it allows you to develop programs using well-defined *control structures* (you will learn about *control structures* in the articles to come), and provides *modularity* (breaking the task into multiple sub tasks that are simple enough to understand and to reuse).

C is often called a **middle-level language** because it combines the best elements of low-level or machine language with high-level languages.

n **C provides:**

- u **Efficiency, high performance and high quality s/ws**
- u **flexibility and power**
- u **many high-level and low-level operations → middle level**
- u **Stability and small size code**
- u **Provide functionality through rich set of function libraries**
- u **Gateway for other professional languages like C → C++ → Java**

n **C is used:**

- u **System software Compilers, Editors, embedded systems**
- u **data compression, graphics and computational geometry, utility programs**
- u **databases, operating systems, device drivers, system level routines**

- u **there are zillions of lines of C legacy code**
- u **Also used in application programs**
- n There is a small, fixed number of keywords, including a full set of flow of control primitives: for, if/else, while, switch, and do/while. There is one namespace, and user-defined names are not distinguished from keywords by any kind of sigil.
- n There are a large number of arithmetical and logical operators, such as +, +=, ++, &, ~, etc.
- n More than one assignment may be performed in a single statement.
- n Function return values can be ignored when not needed.
- n Typing is static, but weakly enforced: all data has a type, but implicit conversions can be performed; for instance, characters can be used as integers.
- n Declaration syntax mimics usage context. C has no "define" keyword; instead, a statement beginning with the name of a type is taken as a declaration. There is no "function" keyword; instead, a function is indicated by the parentheses of an argument list.
- n User-defined (typedef) and compound types are possible.
 - u Heterogeneous aggregate data types (struct) allow related data elements to be accessed and assigned as a unit.
 - u Array indexing is a secondary notion, defined in terms of pointer arithmetic. Unlike structs, arrays are not first-class objects; they cannot be assigned or compared using single built-in operators. There is no "array" keyword, in use or definition; instead, square brackets indicate arrays syntactically, e.g. month[11].
 - u Enumerated types are possible with the enum keyword. They are not tagged, and are freely interconvertible with integers.
 - u Strings are not a separate data type, but are conventionally implemented as null-terminated arrays of characters.
- n Low-level access to computer memory is possible by converting machine addresses to typed pointers.
- n Procedures (subroutines not returning values) are a special case of function, with an untyped return type void.
- n Functions may not be defined within the lexical scope of other functions.
- n Function and data pointers permit *ad hoc* run-time polymorphism.
- n A preprocessor performs macro definition, source code file inclusion, and conditional compilation.
- n There is a basic form of modularity: files can be compiled separately and linked together, with control over which functions and data objects are visible to other files via static and extern attributes.

n Complex functionality such as I/O, string manipulation, and mathematical functions are consistently delegated to library routines.
C does not include some features found in newer, more modern high-level languages, including object orientation and garbage collection

Features of C Language

1 . Low Level Features :

1. C Programming provides low level features that are generally provided by the Lower level languages. C is Closely Related to Lower level Language such as “**Assembly Language**”.
2. It is easier to write assembly language codes in C programming.

2 . Portability :

1. C Programs are portable i.e they can be run on any Compiler with Little or no Modification
2. Compiler and Preprocessor make it Possible for C Program to run it on Different PC

3 . Powerful

1. Provides Wide verity of ‘**Data Types**’
2. Provides Wide verity of ‘**Functions**’
3. Provides useful Control & Loop Control Statements

4 . Bit Manipulation

1. C Programs can be manipulated using bits. We can perform different operations at bit level. We can manage memry representation at bit level. [Eg. We can use Structure to manage Memory at Bit Level]
2. It provides wide verity of bit manipulation Operators. We have bitwise operators to manage Data at bit level.

5 . High Level Features :

1. It is more User friendly as compare to Previous languages. Previous languages such as BCPL,Pascal and other programming languages never provide such great features to manage data.
2. Previous languages have there **pros and cons** but C Programming collected all useful features of previous languages thus C become **more effective language**.

6 . Modular Programming

1. **Modular programming** is a software design technique that increases the extent to which software is composed of separate parts, called **modules**
2. C Program Consist of Different Modules that are integrated together to form complete program

7 . Efficient Use of Pointers

1. Pointers has direct access to memory.

2. C Supports efficient use of pointer .

8 . More Efficient

1. **General Purpose Programming Language:** it is general purpose programming language. It is usually called “system programming language” but equally suited to writing a variety of applications.
2. **Middle Level:** As a middle level language it bridges elements of high level language with the functionality of assembly language.
3. **Structured Programming:** C is very suited for structured programming. The programmers can easily divide a problem into a number of modules or functions.
4. **Simplicity:** C is simple to use because of its structured approach. It has a wide collection of inbuilt functions, keywords, operators and data types.
5. **Portability:** This refers to the ability of a program to run in different environments. With the availability of compilers for almost all operating systems and hardware platforms, it is easy to write code on one system which can be easily ported to another.
6. **Wide Acceptability:** C is widely popular in the software industry. Its importance is not entirely derived from its use as primary development language but also because of its use as an interface language to some of the visual languages.
7. **Flexibility:** C Language combines the convenience and portable nature of a high level language with the flexibility of a low level language. It can be interfaced readily to other programming languages.
8. **Efficient Compilation and Execution:** The process of compilation and execution of programs is quite fast in C Language as compared to other languages like BASIC (**B**eginner’s **A**ll purpose **S**ymbolic **I**nstruction **C**ode) and FORTRAN (**FOR**mula **TRAN**slator).
9. **Modularity:** C Language programs can be divided into small modules with the help of functions which help in increasing understanding of the programs.
10. **Clarity:** The features like keywords, in-built functions and structures help to improve the clarity and hence understanding of the program.
11. **High Availability:** The software of C Language is readily available in market and can be easily installed on the computer.
12. **Easy Debugging:** The syntax errors can be easily detected by the C compiler. The error is displayed with line number of the code and the error message.
13. **Memory Management:** Various memory management in-built functions are available in C language which helps to save memory and

hence improve efficiency of the program. e.g. malloc(),alloc() and calloc().

14. **Recursion:** Recursion is a technique in which the function calls itself again and again until a condition is achieved.
15. **Rich set of Library Functions:** C has a rich set of library functions. These are the functions that provide readymade functionality to the users. It also supports graphic programming.

Facts about C

- C was invented to write an operating system called UNIX.
- C is a successor of B language which was introduced around 1970
- The language was formalized in 1988 by the American National Standard Institute (ANSI).
- By 1973 UNIX OS almost totally written in C.
- Today C is the most widely used System Programming Language.
- Most of the state of the art software have been implemented using C
- **Where is C useful?**
- C's ability to communicate directly with hardware makes it a powerful choice for system programmers. In fact, popular operating systems such as Unix and Linux are written entirely in C. Additionally, even compilers and interpreters for other languages such as FORTRAN, Pascal, and BASIC are written in C. However, C's scope is not just limited to developing system programs. It is also used to develop any kind of application, including complex business ones. The following is a partial list of areas where C language is used:
 - Ø Embedded Systems
 - Ø Systems Programming
 - Ø Artificial Intelligence
 - Ø Industrial Automation
 - Ø Computer Graphics
 - Ø Space Research
 - Ø Image Processing
 - Ø Game Programming

• C Language Advantages :

- C Language has list of advantages due to this it is very much popular language around the world and best suitable for the programmer to learn at the first stage of the programming.
-

- **1. Procedure Oriented Language**

- C Language is procedure oriented language, Here user creates procedures or functions to execute their task. Procedure oriented language is very much easy to learn because it follows algorithm to execute your statements. To develop program using procedure oriented language, you need to draw/prepare algorithm and then start converting it into procedure or functions.

-

- **2. Lots of Libraries**

- C Language provides lots of functions which consist of system generated functions and user defined functions. C Compiler comes with list of header files which consist of many general functions which can be used to develop program. while programmer can also create function as per their requirements that is called as user generated/defined function.

-

- **3. Speed of Compilation**

- C compiler produces machine code very fast compared to other language compiler. C compiler can compile around 1000 lines of code in a seconds or two. One more benefit of the C Compiler is that it also optimize the code for faster execution.

-

- **4. Easy to Learn (Syntax is near to English Language)**

- C Language syntax is very easy to understand. It uses keyword like if, else, goto, switch, goto, main, etc. This kind of keyword we all are using in our day to day life to convey meaning or to get some decisions.

-

- **5. Portable**

- C Language setup is around 3-5 MB. So you can carry this language in your Floppy Drive or Pen Drive. Its very easy to install and operate, Again its output is exe file which can be executed in any computer without any other framework / software.

- **C Language Disadvantages :**

-

- Every coin has two sides, as C Language has also some disadvantages. C Language has not any major disadvantages but some features is missing in the C Language, obviously that's why C Language is very much powerful now.

-

- **1. Object Oriented Programming Features (OOPS)**

- Object Oriented Programming Features is missing in C Language, You have to develop your program using procedure oriented language only.

-
- **2. Run Time Type Checking is Not Available**
- In C Language there is no provision for run time type checking, for example i am passing float value while receiving parameter is of integer type then value will be changed, it will not give any kind of error message.
-
- **3. Namespace Feature**
- C does not provides namespace features, so you can't able to use the same variable name again in one scope. If namespace features is available then you can able to reuse the same variable name.
-
- **4. Constructor and Destructor is not available:**
- C does not provides object oriented features, so it don't have Constructor and Destructor features. Constructor and Destructor is used to construct object and destroy object. So in C Language you have to implement manually construction and destruction of the variable using function or by other means.

Variables

Variables

Variables are means for location in memory used by a program to store data. The size of that block depends upon the range over which the variable is allowed to vary.

For example, on personal computer the size of an integer variable is two bytes, and that of a long integer is four bytes.

A variable region is temporarily remember a number or string value, such as covered by the program. To identify the variables, you have a name unique to every single variable. This is called a variable name. Before using a variable, use variables to what is called a variable declaration that you have to reveal the names and data types that can be stored in the variable variable.

The format for declaring a variable in C.
[Storage-class] type data variable name [= initial value];

Storage class and the initial value can be omitted.

The same data type and storage class variable can be declared, separated by commas.

[Storage-class] type data variable name [= initial value] variable [= initial value] variable [= initial value];

In C the size of a variable type such as an integer need not be the same on all types of machines. When we declare a variable we inform the compiler of two things, the name of the variable and the type of the variable. For example, we declare a variable of type character with the name *i* by writing:
`char i;`

On seeing the "char" part of this statement the compiler sets aside one bytes of memory to hold the value of the character. It also sets up a symbol table. In that table it adds the symbol *i* and the relative address in memory where those one byte was set aside. Thus, later if we write: `i = 'x';` we expect that,at run time when this statement is executed, the value 'x' will be placed in that memory location reserved for the storage of the value of *i*.

- Variable names correspond to locations in the computer's memory
- Every variable has a name, a type, a size and a value
- Whenever a new value is placed into a variable (through **scanf**, for example), it replaces (and destroys) the previous value
- Reading variables from memory does not change them
- A visual representation

integer1

45

All variables must be declared before we use them in C program, although certain declarations can be made implicitly by content. A declaration specifies a type, and contains a list of one or more variables of that type as follows:

`type variable_list;`

Here, type must be a valid C data type including char, int, float, double, or any user defined data type etc., and variable_list may consist of one or more identifier names separated by commas. Some valid variable declarations are shown here:

the names of variables, functions, labels and other user defined functions are called as identifiers.

IDENTIFIER	VARIABLE
All identifiers are not variables	All variables are identifiers
Identifier may not have any memory unless it is a variable	All variables have memory
Mentioning the type of an identifier is not needed unless it is a variable.	Type of the variable must be defined.

Different types of errors:

Error is a mistake or bug, which can be associated with programmer in program, common programming errors in C can be classified into 3 types:

1. compilation errors.(syntax errors)
2. linker errors.
3. logical errors.

Compilation(syntax) errors: these are raised when we compile the program and can be located and corrected easily. Most common compilation or syntax errors are.

- ?? Undefined variable
- ?? Redclaration of variable
- ?? Unterminated string character
- ?? Missing semicolon ;
- ?? Function call missing (,;")
- ?? Function should have prototype.

Linker errors: the program must be linked to the 'C' library. If it fails in such case these errors are raised . most common errors are:

- ?? Unable to link cos.obj
- ?? Undefined symbol

Logical (run time) errors: these are raised because of the due to logical inefficiency. We can know them when program gets executed. It is very difficult to locate them, most common logical errors are:

- ?? Divide by zero
- ?? Floating point error
- ?? Null value
- ?? Garbage result in printing.

A Simple C Program

Every C program must have one special function `main ()`. This is the point where execution begins when the program is running. We will see later that this does not have to be the first statement in the program, but it must exist as the entry point. The group of statements defining the `main ()` enclosed in a pair of braces `{ }` are executed sequentially. Each expression statement must end with a semicolon. The closing brace of the main function signals the end of the program. The main function can be located anywhere in the program but the general practice is to place it as the first function.

Here is an elementary C program.

```
main ()  
{  
}
```

There is no way to simplify this program, or to leave anything out. Unluckily, the program does not do anything. Following the "main" program name is a couple of parentheses, which are an indication to the compiler that this is a function. The 2 curly brackets `{ }`, properly called braces, are used to specify the limits of the program itself. The actual program statements go between the 2 braces and in this case, there are no statements because the program does absolutely nothing. You will be able to compile and run this program, but since it has no executable statements, it does nothing. Keep in mind however, that it is a legal C program.

`main ( )` function should return zero or one. Turbo C accepts both `int` and `void main ( )` and Turbo C coders use both `int` and `void main ( )` in their programs. But in my belief, `void main ( )` is not a standard usage. The reason is, whenever a program gets executed it returns an integer to the OS. If it returns 'zero', the program is executed successfully. Otherwise it means the program has been ended with error. So `main ( )` shouldn't be declared as `void`.d as `void`.

`main( )` should be declared as

```
int main()  
{  
.....
```

```

.....
return 0;
}

```

.For a much more interesting program, load the program

```

int main ()
{
    printf ("Welcome to C language");
    return 0;
}

```

and display it on your monitor. It is same as the previous program except that it has one executable statement between the braces.

The executable statement is another function. Again, we won't care about what a function is, but only how to use this one. In order to output text to the monitor, it's placed within the function parentheses and bounded by quotes. The end result is that whatever is included between the quotes will be showed on the monitor when the program is run.

Notice the semi-colon; at the end of the line. C uses a semi-colon as a statement terminator, so semi-colon is required as a signal to the compiler that this line is complete. This program is also executable, so you'll be able to compile and run it to see if it does what you think it should. With some compilers, you may get an error message while compiling, indicating that the function `printf ()` should have a prototype.

```

#include<stdio.h>
#include<conio.h>
int main ()
{
    printf ("Welcome to C language");
    return 0;
}

```

Here you'll be able to see `#include` at the beginning of the program. It is a pre-processor directive. It's not a part of our program; it's an instruction to the compiler to make it do something. It says the C compiler to include the contents of a file, in this case the system file `stdio.h`. This is the name of the standard library definition file for all Standard Input Output. Your program will almost certainly want to send stuff to the screen and read things from the keyboard. `stdio.h` is the name of the file in which the functions that we want to use are defined. A function is simply a group of related statements that we can use later. Here the function we used is `printf`. To use `printf` correctly C needs to know what it looks like, i.e. what things it can work on and what value it returns. The actual code which performs the `printf` will be tied in later by the linker. Note that without the definition of what `printf` looks like the

compiler makes a guess when it sees the use of it. This can lead to the call failing when the program runs, a common cause of programs crashing.

The <> characters around the name tell C to look in the system area for the file stdio.h. If I had given the name "abc.h" instead it would tell the compiler to look in the current directory. This means that I can arrange libraries of my own routines and use them in my programs.

Imagine you run above program and then change it and run it again you may find that the previous output is still stucked there itself, at this time `clrscr();` would clear the previous screen. One more thing to remember while using `clrscr()` is that it should be called only after the variable declaration, like

```
int p,q,r;
clrscr()
```

Here is an example of minimal C program that displays the string Welcome to C language (this famous example included in all languages moreover been done originally in C in 1978 from the creators of the language, Brian Kernighan and Dennis Ritchie) Example

```
#include<stdio.h>
#include<conio.h>
int main ()
{
    clrscr();
    printf ("Welcome to C language");
    return 0;
}
```

When you execute above program you won't see 'Welcome to C language' on the console because the screen will just flash and go away .If you want to see the line you can use `getch()` function just below the `printf()` statement. Actually it waits until a key is pressed.

#include<stdio.h>

With this line of code we include a file called stdio.h. (Standard Input/Output header file). This file lets us use certain commands for input or output which we can use in our program. (Look at it as lines of code commands) that have been written for us by someone else). For instance it has commands for input like reading from the keyboard and output commands like printing things on the screen.

int main()

The int is what is called the return value (in this case of the type integer).

Where it used for will be explained further down. Every program must have a `main()`. It is the starting point of every program. The round brackets are there for a

reason, in a later tutorial it will be explained, but for now it is enough to know that they have to be there.

{

The two curly brackets (one in the beginning and one at the end) are used to group all commands together. In this case all the commands between the two curly brackets belong to `main()`. The curly brackets are often used in the C language to group commands together. (To mark the beginning and end of a group or function.).

printf("Hello World\n");

The `printf` is used for printing things on the screen, in this case the words: Hello World. As you can see the data that is to be printed is put inside round brackets. The words Hello World are inside inverted commas, because they are what is called a string. (A single letter is called a character and a series of characters is called a string). Strings must always be put between inverted commas. The `\n` is called an escape sequence. In this case it represents a newline character. After printing something to the screen you usually want to print something on the next line. If there is no `\n` then a next `printf` command will print the string on the same line.

Commonly used escape sequences are:

- `\n` (newline)
- `\t` (tab)
- `\v` (vertical tab)
- `\f` (new page)
- `\b` (backspace)
- `\r` (carriage return)

After the last round bracket there must be a semi-colon. The semi-colon shows that it is the end of the command. (So in the future, don't forget to put a semi-colon if a command ended).

return 0;

When we wrote the first line "`int main()`", we declared that `main` must return an integer `int main()`. (`int` is short for integer which is another word for number). With the command `return 0;` we can return the value null to the operating system. When you return with a zero you tell the operating system that there were no errors while running the program. (Take a look at [the C tutorial – variables and constants](#) for more information on integer variables).

```
// C Basics example Program
#include <stdio.h>

int main()
{
```

```
/* Our first simple C basic program */  
printf("Hello World! ");  
return 0;  
}
```

Output:

Hello World!

Let us see the parts of the program line by line.

S.no	Command	Explanation
1	#include<stdio.h>	This is a preprocessor command that includes standard input output header file(stdio.h) from the C library before compiling a C program
2	int main()	This is the main function from where execution of any C program begins
3	/* some comments */	whatever is given inside /* */ this command won't be considered by C program for compilation and execution.
4	printf("Hello World! ");	printf command prints the output onto the screen
5	return 0;	This command terminates main function then returns 0

The basic building block of a C program is the function. Every C program is a collection of one or more functions. Functions are made of variable declarations and statements, or complex commands, and are surrounded by curly brackets ('{' and '}'). One and only one of the functions in a program must have the name main. This function is always the starting point of a C program, so the simplest C program is a single function

definition:

```
main ()
```

```
{  
}
```

The parentheses '()' that follow the name of the function must be included.

This is how C

distinguishes functions from ordinary variables.

The function main does not need to be at the top of a program, so a C program does not

necessarily start at line 1, but wherever the function called main is located.

The function

main cannot be called, or started, by any other function in the program. Only the operating

system can call main; this is how a C program is started.

The next most simple C program is perhaps a program that starts, calls a function that

does nothing, and then ends.

Compile and execute C program:

- Compilation is the process of converting a C program which is user readable code into machine readable code which is 0's and 1's.
- This compilation process is done by a compiler which is an inbuilt program in C.
- As a result of compilation, we get another file called executable file. This is also called as binary file.
- This binary file is executed to get the output of the program based on the logic written into it.

Steps to compile & execute a C program:

Step1 Type the above C basic program in a text editor and save as "sample.c".

Step2 To compile this program, open the command prompt and goto the directory where you have saved this program and type "cc sample.c" or

"gcc sample.c".

Step3 If there is no error in above program, executable file will be generated in the name of "a.out".

Step4 To run this executable file, type "./a.out" in command prompt.

Step5 You will see the output as shown in the above basic program.

There are three steps in executing a c++ program: Compiling, Linking and Running the program. The c++ programs have to be typed in a compiler. All the programs discussed in the book will be compiled on turbo c++ compiler. The turbo c++ compiler comes with an editor to type and edit c++ program.

After typing the program the file is saved with an extension .cpp. This is known as source code. The source code has to be converted to an object code which is understandable by the machine. This process is known as compiling the program. You can compile your program by selecting compile from compile menu or press Alt+f9. After compiling a file with the same name as source code file but with extension .obj. is created.

Second step is linking the program which creates an executable file .exe (filename same as source code) after linking the object code and the library files (cs.lib) required for the program. In a simple program, linking process may involve one object file and one library file. However in a project, there may be several smaller programs. The object codes of these programs and the library files are linked to create a single executable file. Third and the last step is running the executable file where the statements in the program will be executed one by one.

When you execute the program, the compiler displays the output of the program and comes back to the program editor. To view the output and wait for user to press any key to return to the editor, type getch() as the last statement in the program. Getch() is an inbuilt predefined library function which inputs a character from the user through standard input. However you should include another header file named conio.h to use this function. Conio.h contains the necessary declarations for

using this function. The include statement will be similar to iostream.h.

Compiling and Linking

During compilation, if there are any errors that will be listing by the compiler. The errors may be any one of the following

1.Syntax error

This error occurs due to mistake in writing the syntax of a c++ statement or wrong use of

reserved words, improper variable names, using variables without declaration etc. Examples are :

missing semi colon or paranthesis, type integer for int datatype etc.

Appropriate error message and

the statement number will be displayed. You can see the statement and make correction to the

program file, save and recompile it.

2. Logical error

This error occurs due to the flaw in the logic. This will not be identified by the compiler.

However it can be traced using the debug tool in the editor. First identify the variable which you

suspect creating the error and add them to watch list by selecting Debug ->Watches->Add watch.

Write the variable name in the watch expression. After adding all the variables required to the watch

list, go to the statement from where you want to observe. If you are not sure, you can go to the first

statement of the program. Then select Debug ->Toggle Breakpoint (or press ctrl + f8). A red line will

appear on the statement. Then Run the program by selecting Ctrl + f9 or Run option from run menu.

The execution will halt at the statement where you had added the breakpoint. The watch variables and

their values at that point of time will be displayed in the bottom in the watch window. Press F8 to

execute the next statement till you reach the end of the program. In this way you can watch closely

the values in the watch variables after execution of each and every statement in the program. If you

want to exit before execution of the last statement press Ctrl + Break. To remove the breakpoint in the

program go to the statement where you have added breakpoint select

Debug ->Toggle Breakpoint (or press ctrl + f8).Select Debug -> watch ->remove watches to remove the variables in the watch list. This tool helps in knowing the values taken by the variable at each and every step. You can compare the expected value with the actual value to identify the error.

3. Linker error

This error occur when the files during linking are missing or misspelt

4. Runtime error

This error occurs if the program encounters division by zero, accessing a null pointer etc during execution of the program.

C/C++ Program Compilation

In this chapter we begin by outlining the basic processes you need to go through in order to compile your C (or C++) programs. We then proceed to formally describe the C compilation model and also how C supports additional libraries.

Creating, Compiling and Running Your Program

The stages of developing your C program are as follows. (See Appendix □ and exercises for more info.)

Creating the program

Create a file containing the complete program, such as the above example. You can use any ordinary editor with which you are familiar to create the file. One such editor is *textedit* available on most UNIX systems.

The filename must by convention end ".c" (full stop, lower case c), *e.g. myprog.c* or *progtest.c*. The contents must obey C syntax. For example, they might be as in the above example, starting with the line `/* Sample ....` (or a blank line preceding it) and ending with the line `} /* end of program */` (or a blank line following it).

Compilation

There are many C compilers around. The `cc` being the default Sun compiler. The GNU C compiler `gcc` is popular and available for many platforms. PC users may also be familiar with the Borland `bcc` compiler.

There are also equivalent C++ compilers which are usually denoted by `CC` (*note* upper case `CC`. For example Sun provides `CC` and GNU `GCC`. The GNU compiler is also denoted by `g++`

Other (less common) C/C++ compilers exist. All the above compilers operate in essentially the same manner and share many common command line options. Below and in Appendix □ we list and give example uses many of the

common compiler options. However, the **best** source of each compiler is through the online manual pages of your system: *e.g.* `man cc`.

For the sake of compactness in the basic discussions of compiler operation we will simply refer to the `cc` compiler -- other compilers can simply be substituted in place of `cc` unless otherwise stated.

To Compile your program simply invoke the command `cc`. The command must be followed by the name of the (C) program you wish to compile. A number of compiler options can be specified also. We will not concern ourselves with many of these options yet, some useful and often essential options are introduced below -- See Appendix [□](#) or online manual help for further details.

Thus, the basic compilation command is:

```
cc program.c
```

where ***program.c*** is the name of the file.

If there are obvious errors in your program (such as mistypings, misspelling one of the key words or omitting a semi-colon), the compiler will detect and report them.

There may, of course, still be logical errors that the compiler cannot detect. You may be telling the computer to do the wrong operations.

When the compiler has successfully digested your program, the compiled version, or executable, is left in a file called ***a.out*** or if the compiler option ***-o*** is used : the file listed after the ***-o***.

It is more convenient to use a ***-o*** and filename in the compilation as in

```
cc -o program program.c
```

which puts the compiled program into the file `program` (or any file you name following the `"-o"` argument) **instead** of putting it in the file `a.out` .

Running the program

The next stage is to actually run your executable program. To run an executable in UNIX, you simply type the name of the file containing it, in this case ***program*** (or ***a.out***)

This executes your program, printing any results to the screen. At this stage there may be run-time errors, such as division by zero, or it may become evident that the program has produced incorrect output.

If so, you must return to edit your program source, and recompile it, and run it again.

The C Compilation Model

We will briefly highlight key features of the C Compilation model (Fig. [2.1](#)) here.

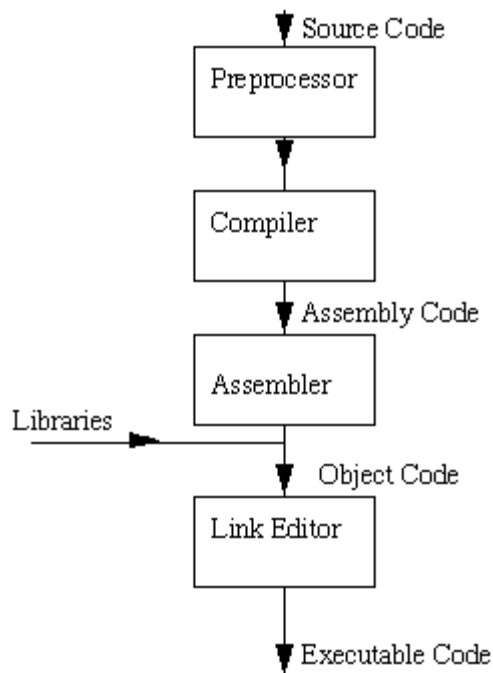


Fig. 2.1 The C Compilation Model

The Preprocessor

We will study this part of the compilation process in greater detail later (Chapter [13](#)). However we need some basic information for some C programs.

The Preprocessor accepts source code as input and is responsible for

- removing comments
- interpreting special ***preprocessor directives*** denoted by #.

For example

- **#include** -- includes contents of a named file. Files usually called **header** files. *e.g*
 - **#include <math.h>** -- standard library maths file.
 - **#include <stdio.h>** -- standard library I/O file
- **#define** -- defines a symbolic name or constant. Macro substitution.
 - **#define MAX_ARRAY_SIZE 100**

C Compiler

The C compiler translates source to assembly code. The source code is received from the preprocessor.

Assembler

The assembler creates object code. On a UNIX system you may see files with a .o suffix (.OBJ on MSDOS) to indicate object code files.

Link Editor

If a source file references library functions or functions defined in other source files the **link editor** combines these functions (with main()) to create an executable file. External Variable references resolved here also. ***More on this later*** (Chapter [34](#)).

Printf and Scanf

The printf() and scanf() family of functions are part of the standard C library, and make it easy to take input from the user, and display data to the user. Learning how to effectively use these functions can open up a whole world of possibilities when building new projects.

The formatting placeholders in scanf are more or less the same as that in printf, its reverse function.

There are rarely constants (i.e. characters that are not formatting placeholders) in a format string, mainly because a program is usually not designed to read known data. The exception is one or more whitespace characters, which discards all whitespace characters in the input.

The printf() function is used to print the character, string, float, integer, octal and hexadecimal values onto the output screen.

To display the value of an integer variable, we use printf statement with the %d format specifier.

Similarly %c for character, %f for float variable, %s for string variable, %lf for double, %x for hexadecimal variable.

A call to printf looks like this:

```
printf( "format-string", expression, ... );
```

in the first printf statement \t requests for the tab displacement on the screen the argument %d tells the compiler that the value of num should be printed as decimal integer. \n causes the new output to start from new line.

In second printf statement %5.2f tells the compiler that the output must be in floating point, with five places in all and two places to the right of the decimal point.

scanf() function is used to read a character, string, numeric data from keyboard.

A call to scanf looks like this:

```
scanf( "conversion-string", &variable, ... );
```

The data from the keyboard is received by scanf function. In the above format, the & (ampersand) symbol before each variable name is an operator that specifies the address of variable name.

Some of the most commonly used placeholders follow:

- %d : Scan an integer as a signed decimal number.
- %i : Scan an integer as a signed number. Similar to %d, but interprets the number as hexadecimal when preceded by 0x and octal when preceded by 0. For example, the string 031 would be read as 31 using %d, and 25 using %i. The flag h in %hi indicates conversion to a short and hh conversion to a char.

- %u : Scan for decimal unsigned int (Note that in the C99 standard the input value minus sign is optional, so if a negative^[clarification needed] number is read, no errors will arise and the result will be the two's complement, effectively ignoring the negative sign in most cases. See strtoul().^[not in citation given]) Correspondingly, %hu scans for an unsigned short and %hhu for an unsigned char.
- %f : Scan a floating-point number in normal (fixed-point) notation.
- %g, %G : Scan a floating-point number in either normal or exponential notation. %g uses lower-case letters and %G uses upper-case.
- %x, %X : Scan an integer as an unsigned hexadecimal number.
- %o : Scan an integer as an octal number.
- %s : Scan a character string. The scan terminates at whitespace. A null character is stored at the end of the string, which means that the buffer supplied must be at least one character longer than the specified input length.
- %c : Scan a character (char). No null character is added.
- whitespace: Any whitespace characters trigger a scan for zero or more whitespace characters. The number and type of whitespace characters do not need to match in either direction.
- %lf : Scan as a double floating-point number.
- %Lf : Scan as a long double floating-point number.

Escape Sequence Characters (Backslash Character Constants) in C:

C supports some special escape sequence characters that are used to do special tasks.

These are also called as 'Backslash characters'.

Some of the escape sequence characters are as follow:

Character Constant	Meaning
\n	New line (Line break)
\b	Backspace
\t	Horizontal Tab
\f	Form feed
\a	Alert (alerts a bell)
\r	Carriage Return

<code>\v</code>	Vertical Tab
<code>\?</code>	Question Mark
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote
<code>\\</code>	Backslash
<code>\0</code>	Null

Code	Meaning
<code>\b</code>	Backspace
<code>\f</code>	Form feed
<code>\n</code>	New line
<code>\r</code>	Carriage return
<code>\t</code>	Horizontal tab
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\\</code>	Backslash
<code>\v</code>	Vertical tab
<code>\a</code>	Alert
<code>\?</code>	Question mark
<code>\N</code>	Octal constant (N is an octal constant)
<code>\xN</code>	Hexadecimal constant (N is a hexadecimal constant)

C Character Set :

Whenever we write any C program then it consists of different statements. Each C Program is set of statements and each statement is set of different c programming lexims. In C Programming each and every character is considered as single lexim. i.e [Basic Lexical Element]

Character Set Consists Of -

Types	Character Set
Lowercase Letters	a-z
Uppercase Letters	A to Z
Digits	0-9
Special Characters	!@#\$%^&*
White Spaces	Tab Or New line Or Space

Special Characters:

C language contains the following special character in association with the letters and digits.

Symbol	Meaning
~	Tilde
!	Exclamation mark
#	Number sign
\$	Dollar sign
%	Percent sign
^	Caret
&	Ampersand
*	Asterisk
(	Left parenthesis
)	Right parenthesis
_	Underscore
+	Plus sign
	Vertical bar
\	Backslash
`	Apostrophe
-	Minus sign
=	Equal to sign
{	Left brace
}	Right brace
[	Left bracket
]	Right bracket
:	Colon
"	Quotation mark
;	Semicolon
<	Opening angle bracket
>	Closing angle bracket
?	Question mark
,	Comma
.	Period
/	Slash

Tokens : The smallest individual units of a C- program are called Tokens. **Key words, Identifiers, Constants, Operators, Delimiters.**
Key words : have a predefined meaning and these meanings cannot be changed. All keywords must be written in small letters (except additional c99 keywords).

Keywords

Keywords are standard identifiers that have standard predefined meaning in C. Keywords are all lowercase, since uppercase and lowercase characters are not equivalent it's possible to utilize an uppercase keyword as an identifier but it's not a good programming practice.

Points to remember

1. Keywords can be used only for their intended purpose.
2. Keywords can't be used as programmer defined identifier.
3. The keywords can't be used as names for variables

void	char	int	float	double	signed	unsigned	short
long	auto	static	register	const	struct	union	if
else	goto	for	continue	switch	case	break	default
do	while	sizeof	volatile	enum	extern	typedef	return

Lvalues and Rvalues in C

There are two kinds of expressions in C:

1. **lvalue**: An expression that is an **lvalue** may appear as either the left-hand or right-hand side of an assignment.
2. **rvalue**: An expression that is an **rvalue** may appear on the right- but not left-hand side of an assignment.

Variables are lvalues and so may appear on the left-hand side of an assignment. Numeric literals are rvalues and so may not be assigned and cannot appear on the left-hand side. Following is a valid statement:

```
int g = 20;
```

But following is not a valid statement and would generate compile-time error:

```
10 = 20;
```

Defining Constants

There are two simple ways in C to define constants:

1. Using **#define** preprocessor.
2. Using **const** keyword.

The #define Preprocessor

Following is the form to use #define preprocessor to define a constant:

TUTORIALS POINT Simply Easy Learning Page 19

#define identifier value

Following example explains it in detail:

```
#include <stdio.h>
```

```
#define LENGTH 10
```

```
#define WIDTH 5
```

```
#define NEWLINE '\n'
```

```
int main()
```

```
{
```

```
int area;
```

```
area = LENGTH * WIDTH;
```

```
printf("value of area : %d", area);
```

```
printf("%c", NEWLINE);
```

```
return 0;
```

```
}
```

When the above code is compiled and executed, it produces following result:

value of area : 50

The const Keyword

You can use **const** prefix to declare constants with a specific type as follows:

const type variable = value;

Following example explains it in detail:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
const int LENGTH = 10;
```

```
const int WIDTH = 5;
```

```
const char NEWLINE = '\n';
```

```
int area;
```

```
area = LENGTH * WIDTH;
```

```
printf("value of area : %d", area);
```

```
printf("%c", NEWLINE);
```

```
return 0;
```

```
}
```

When the above code is compiled and executed, it produces following result:

value of area : 50

Note that it is a good programming practice to define constants in CAPITALS.

Identifiers : Identifiers are the names that are given to various program elements such as variables, symbolic constants and functions, Variable or function identifier that is called a symbolic constant name.

Rules for define identifiers :

- a) First character must be alphabetic character or under score
- b) Second character onwards alphabetic character of digit or under score.
- c) First 63 characters of an identifier are significant.
- d) Cannot duplicate a key word.
- e) May not have a space or any other special symbol except under score.
- f) C – language is Case-sensitive.

Whitespace in C

A line containing only whitespace, possibly with a comment, is known as a blank line, and a C compiler totally ignores it.

Whitespace is the term used in C to describe blanks, tabs, newline characters and comments. Whitespace separates one part of a statement from another and enables the compiler to identify where one element in a statement, such as int, ends and the next element begins. Therefore, in the following statement:

```
int age;
```

There must be at least one whitespace character (usually a space) between int and age for the compiler to be able to distinguish them. On the other hand, in the following statement

```
fruit = apples + oranges; // get the total fruit
```

No whitespace characters are necessary between fruit and =, or between = and apples, although you are free to include some if you wish for readability purpose.

Constants : fixed values that do not change during execution of a program.

Boolean constants : 0 (false) and 1 (true)

Character constants :

only one character enclosed between two single quotes (except escape characters).

wide character type - wchar_t - for Unicode characters.

Data Types

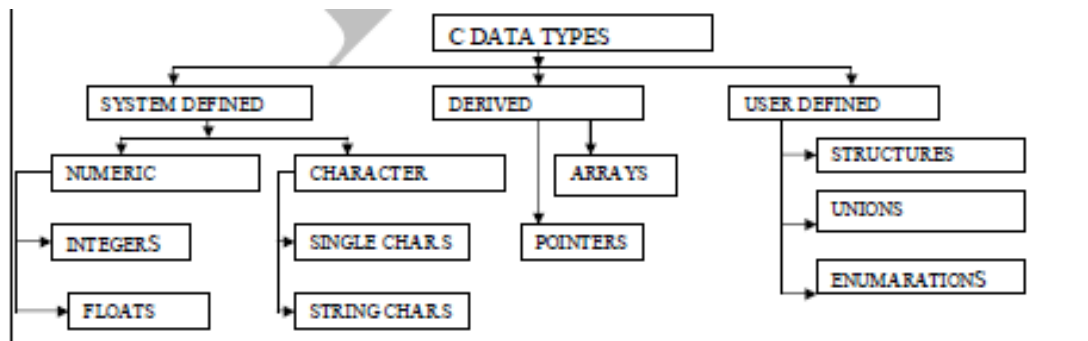
Data Types

Data types in C :

Data type determines the type of data a variable will hold. If a

variable 'x' is declared as 'int'. it means x can hold only integer values.

Every variable which is used in the program must be declared as what data-type it is.



System defined data types: these data types can be defined by C compiler, these are two types:

Numeric data types : these are two types: integers and characters.

Character data types: these are two types single characters and string characters.

Derived data types: these data types can be declared by user by deriving the system defined data type concepts. These are two types.

Arrays : set of similar data elements are called as arrays.

Ex: int a[10], char name[20];

Pointers: pointer is a variable which holds the address of similar data variable.

Ex: int a=5,*p; -> *p is a pointer variable declared with * character.

User defined data types: these data types can be defined by user, these are three types.

Structures : group of deferent data items combined together are called as structures. All the data items can stored in individual memory locations.

Unions : group of deferent data items combined together are called as unions. All the data items can share unique memory location.

Enumerations: these data types allows t declare string constants with integer equalant values.

In the C programming language, data types refers to an extensive system used for declaring variables or functions of different types. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted.

C offers a standard, minimal set of basic data types. Sometimes these are called "primitive" types. A lot of complex data structures can be developed from these basic data types. The C language defines 4 fundamental data types:

character

integer

floating-point

double

and

floating-point

This data types are declared using the keywords char,int,float and double

respectively. Typical memory requirements of the basic data types are given below

Data type	Meaning	Size (byte)	Minimal range
char	Character	1	-128 to 127
int	Integer	2	-32,768 to 32,768
float	Single precision real number	4	3.4E-38 to 3.4E+38
double	Double precision real number	8	1.7E-308 to 1.7E+308
void	Valueless	0	

The size and range of these data types may vary among processor types and compilers. Data type qualifiers modify the behavior of variable type to which they are applied. Data type qualifiers can be classified into two types.

1. size qualifiers
2. sign qualifiers

Size qualifiers:

Size qualifiers alter the size of the basic data types. There are two size qualifiers that can be applied to integer: short and long.

The minimum size of short int is 16 bit. The size of int must be greater than or equal to that of a short int. The size of long int must be greater than or equal to a short int. The minimum size of a long int is 32 bits.

Sign qualifiers:

The keywords signed and unsigned are the two sign qualifiers that specify whether a variable can hold both -ve and +ve numbers, or only +ve numbers. These qualifiers can be applied to the data types int and char only.

Example: unsigned int I;

The following table shows all valid data type combinations supported by C, along with their minimal ranges and typical memory size.

Type	Size (byte)	Minimal range
char	1	-127 to 127
unsigned char	1	0 to 255
signed char	1	-127 to +127
int	2 or 4	-32,767 to 32,767
unsigned int	2 or 4	0 to 65,535
signed int	2 or 4	-32,767 to 32,767
short int	2	-32,767 to 32,767
unsigned short int	2	0 to 65,535
signed short int	2	-32,767 to 32,767
long int	4	-2,147,483,647 to 2,147,483,647
signed long int	4	-2,147,483,647 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295
float	4	3.4E-38 to 3.4E+38 with 6 digits of precision
double	8	1.7E-308 to 1.7E+308 with 10 digits of precision
long double	8	1.7E-308 to 1.7E+308 with 10 digits of precision

char

char is a special integer type designed for storing single characters. The integer value of a char corresponds to an ASCII character. E.g., a value of 65 corresponds to the letter A, 66 corresponds to B, 67 to C, and so on.

As in the table below, unsigned char permits values from 0 to 255,

and signed char permits values from -127 (or -128) to 127. The char type is signed by default on some computers, but unsigned on others the sizes of the char types are as follows.

Type	Bytes	Minimal range
char	1	-127 to 127
unsigned char	1	0 to 255
signed char	1	-127 to 127

int

An integer is a whole number (a number without a fractional part). It can be positive or negative numbers like 1, -2, 3, etc., or zero.

The sizes of the integer variables depend on the hardware and operating system of the computer. On a typical 16-bit system, the sizes of the integer types are as follows.

TYPE	Bytes	Possible Values le Values
int	2 or 4	-32,767 to 32,767
unsigned int	2 or 4	0 to 65,535
signed int	2 or 4	-32,767 to 32,767
short int	2	-32,767 to 32,767
unsigned short int	2	0 to 65,535
signed short int	2	-32,767 to 32,767
long int	4	-2,147,483,647 to 2,147,483,647
signed long int	4	-2,147,483,647 to 2,147,483,647
unsigned long int	4	0 to 4,294,967,295

char

char is a special integer type designed for storing single characters. The integer value of a char corresponds to an ASCII character. E.g., a value of 65 corresponds to the letter A, 66 corresponds to B, 67 to C, and so on.

As in the table below, unsigned char permits values from 0 to 255, and signed char permits values from -127 (or -128) to 127. The char type is signed by default on some computers, but unsigned on others the sizes of the char types are as follows.

Type	Bytes	Minimal range
char	1	-127 to 127
unsigned char	1	0 to 255
signed char	1	-127 to 127

Float

Floating point numbers are numbers with a decimal point. The float type can take large floating point numbers with a small degree of precision (Precision is simply the number of decimal places to which a number can be calculated with accuracy. If a number can be calculated to three decimal places, it is said to have three significant digits.)

Memory size : 4 bytes

Minimal range : $1E-38$ to $1E+38$ with six digit of precision

Double

Double-precision floating point numbers are also numbers with a decimal point. We know that the float type can take large floating point numbers with a small degree of precision but the double-precision double type can hold even larger numbers with a higher degree of precision.

The sizes of the double types are as follows.

<u>Type</u>	<u>Bytes</u>	<u>Minimal range</u>
doubleprecision	8	$1E-38$ to $1E+38$ with 10 digit of precision
long double	8	$1E-38$ to $1E+38$ with 10 digit of precision

Constants

The term constant means that it does not change during the execution of program. In the language C, constant is the data with a constant value that does not change in the program. For example, in the program "100" "3.14" "A" "" "Hello "" and the like, if you write data directly, and constant. Moreover, also called a literal constant. Constant expression is an expression consisting only of constants. There are four basic types of constants in C. They are:

1. <u>Integer</u>	<u>constants</u>
2. <u>Floating-point</u>	<u>constants</u>
3. <u>Character</u>	<u>constants</u>
4. <u>String constants</u>	

Integer and floating-point constants represent numbers. They are often referred to as numeric-type constants. The following rule applies to all numeric type constants:

- Comma and blank spaces cannot be included within the constants.

- Constants can be preceded by a – or + sign, if desired. If either sign does not precede the constant it is assumed to be positive.
- The value of a constant cannot exceed specified minimum and maximum bounds. For each type of constant, these bound vary from one C compiler to another.

Integer constants

Integer constants are whole numbers without any fractional part. Thus integer constants consist of a sequence of digits. Integer constants can be written in three different number systems: Decimal, Octal and Hexadecimal.

A decimal integer constant consists of any combination of digits taken from the set 0 through 9. If the decimal constant contains two or more digits, the first digit must be something other than 0. The following are valid decimal integer constants.

0 1 1234 -786

Floating-point constants

A floating-point constant is a base-10 number that contains either a decimal point or an exponent or both. A floating-point constant can be written in two forms: Factorial form or Exponential form. A floating-point constant in a fractional form must have at least one digit each to the left and right of the decimal point. A floating-point in exponent form consists of a mantissa and an exponent. The mantissa itself is represented as a decimal integer constant or a decimal floating-point constant in fractional form. The letter E or e and the exponent follow the mantissa. The exponent must be a decimal integer. The actual number of digits in the mantissa and the exponent depends on the computer being used.

The following are valid floating-point constants. </1.0 0.1 2E-4 -
0.1555e-4

Character constants

A character constant is a single character, enclosed in single quotation marks.

e.g., 'A' 'B' '1'

Characters are stored internally in computer as coded set of binary digits, which have positive decimal integer equivalents. The value of a character constant is the numeric value of the character in the machine's character set. This means that the value of a character constant can vary from one machine to the next, depending on the character set being used on the particular machine. For example, on ASCII machine the value of 'A' is 65 and on EBCDIC machine it is 193.

String constants

A string constant consists of zero or more character enclosed in quotation marks. Several string constants are given below.

"Welcome to C Programming"

" "

"a+b\n"

"Error\a\a\a"

There is a difference between the constant 'A' and "A" in C, the first 'A' is a character constant while the second "A" is string constant. The notation 'A' is a constant occupying a single byte containing the ASCII code of the character A. The notation "A" on the other hand, is a constant that occupies two bytes one for ASCII code of A and another for the null character with the value 0, that terminates all the string.

Expressions

An expression is a sequence of operators and operands that specifies computation of a value. An expression may consist of single entity or some combination of such entities interconnected by one or more operators. All expression represents a logical connection that's either true or false. Thus logical type expression actually represents numerical quantities.

In C every expression evaluates to a value i.e., every expression results in some value of a certain type that can be assigned to a variable. Some examples of expressions are shown in the table given below.

A+b

3.14*r*r

a*a+2*a*b+b*b

Symbolic constant in c Language

A symbolic constant is name that substitute for a sequence of character that cannot be changed. The character may represent a numeric constant, a character constant, or a string. When the program is compiled, each occurrence of a symbolic constant is replaced by its corresponding character sequence. They are usually defined at the beginning of the program. The symbolic constants may then appear later in the program in place of the numeric constants, character constants, etc., that the symbolic constants represent.

For example

, a **C program** consists of the following symbolic constant definitions.

```
#define PI 3.141593
```

```
#define TRUE 1
```

```
#define FALSE 0
```

#define PI 3.141593 defines a symbolic constant PI whose value is 3.141593. When the program is preprocessed, all occurrences of the symbolic constant PI are replaced with the replacement text 3.141593.

Note that the **preprocessor statements** begin with a #symbol, and are not end with a semicolon. By convention, **preprocessor** constants are written in UPPERCASE.

Declarations

An object **declaration** in C follows the global format:

<class> <type> <name> <initialization> ;

This global format is shared by all the C objects including functions.

Each field differs depending on the object type.

<class> is the **storage class**, and gives information about how the object is allocated and where it is known and then accessible.

<type> gives the basic **type** of the object and is generally completed by the name information.

<name> gives a **name** to the object and is generally an identifier. It may be followed or preceded by additional information to declare complex objects.

<initialization> gives an **initial value** to the object. Depending on how the object is allocated, it may be static information to be built by the linker, or it may be some executable code to set up the variable when it is created. This field is optional and may be omitted.

Each declaration is terminated by the semicolon character ;. To be more convenient, a declaration may contain several occurrences of the pair *<name> <initialization>*, separated by commas. Each variable shares the same *storage class* and the same basic *type*.

Blocks (compound statements)

A block of statements, also called a **compound statement**, is a group of statements that is treated by the compiler as if it were a single statement. Blocks begin with a { symbol, end with a } symbol, and the statements to be executed are placed in between. Blocks can be used any place where a single statement is allowed.

You have already seen an example of a block when writing the function

```
main():  
1      int main()  
2      { // start a block  
3  
4      // multiple statements  
5      int nValue = 0;  
6      return 0;  
7  
8      } // end a block
```

C Tokens:

C tokens includes identifiers, keywords, variables, constants

and operators.

Identifiers : the names of variables, functions, labels and various other user defined items are called as identifiers. Identifiers contains alphabets, digits and underscore. It can't include spaces. It starts with alphabet.

Ex: lcm(), ab_cd, pi, r etc.

Operators

If their nature:

Type	Operators	Meaning
Arithmetic operators	+, -, *, /, %	Addition subtraction multiplication division modulus
Relational operators	>, >=, <, <=, ==, !=	Greater than greater than or equal to less than less than or equal to equal to (comparison) not equal to
Logical operators	&&, , !	AND OR NOT
Assignment operators	=, +=, -=, /=, %=	assignment
Bit wise operators	&, , ^, <<, >>, ~	Bit wise AND Bit wise OR Bit wise XOR left shift right shift complement
Other operators	++, --, sizeof, ? :	Increment decrement sizeof operator ternary conditional

An operator in general, is a symbol that operates on a certain data type. C language is very rich in operators. The commonly used operators include

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Assignment operators
5. Conditional operators
6. Comma operators
7. Unary operators
8. Bitwise Operators

Arithmetic operators

The Arithmetic operators perform Arithmetic operations . The arithmetic operators can operate on any built in data types. A list of arithmetic operator and their meaning are given below.

Operator	Meaning
+	<u>Addition or Unary plus</u>
-	<u>Subtraction or Unary minus</u>
*	<u>Multiplication</u>
/	<u>Division</u>

%

Modulo Division

This table shows the symbols of arithmetic, together with their duties. These operators allow you to write expressions whose evaluation is precisely the treatment of information that made the computer. Arithmetic operators, along with a wide range of features resident in the library of the language used make it possible to perform calculations of all kinds.

Suppose that a and b are integer variables whose values are 100 and 4, respectively. Several arithmetic expressions involving these variables are shown below, together with their resulting values.

Expression	Value
a + b	104
a - b	96
a * b	400
a / b	25
a % b	0

Relational operators

Relational operators are used to compare, logical, arithmetic and character expression. Each of these six relational operators takes two operands. Each of these operators compares their left side with their right side. The whole expression involving the relation operator then evaluate to an integer. It evaluates to 0 if the condition is false and 1 if it is true.

Operator	Meaning
<	<u>Less than</u>
>	<u>Greater than</u>
<=	<u>Less than or equal to</u>
>=	<u>Greater than or equal to</u>
==	<u>Equal to</u>
!=	<u>Not equal to</u>

The concept of comparison certainly implies different meanings depending on the data type of the variables compared. Operators presented in this table are suitable for comparing numeric values or ordinal (individual characters or enumerated values), but not to compare strings or lists of any kind. String comparison requires the use of features designed specifically for this purpose, like `strcmp()` . See [here](#) a description of the string comparison functions offered by C. Note also that these functions do not properly handle diacritics,

it will be necessary to use other methods to compare strings correctly in languages that contain them.

Suppose that a and b are integer variables whose values are 100 and 4, respectively. Several arithmetic expressions involving these variables are shown below, together with their resulting values.

<u>Expression</u>	<u>Interpretation</u>	<u>Value</u>
a<b	False	0
a>b	True	1
a<=b	False	0
a>=b	True	1
a==b	False	0
a!=b	True	1

Logical operators

A Logical operator is used to compare or evaluate logical and relational expression. There are three logical operators in C language. They are

Operator	Meaning
&&	<u>Logical AND</u>
 	<u>Logical OR</u>
!	<u>Logical NOT</u>

Used to calculate the value of logical expressions, ie expressions that the only values that can take operands are True or False. Two versions of these operators. When you simply want to know if an expression is true or false (eg, $x > 7$), operators are used lógicos . These operators do not consider the structure of bits, simply take the value 0 as false and any other as true. There is also another version (the bitwise) imposed by the logical operations to the bits used to internally represent the operands. These are the operators & , | and ~ , respectively.

Suppose that a and b are integer variables whose values are 100 and 4, respectively. Several arithmetic expressions involving these variables are shown below, together with their resulting values.

Expression	Interpretation	Value
(a>b)&&(a==100)	True	1
(b>a) (a>b)	True	1
!(a>b)	False	0

Assignment operator

An Assignment operator is used to form an assignment expression, which assigns the value to an identifier. The most commonly used assignment operator is = . Assignment expressions that make use of this operator are written in the form <Identifier>=expression;

Example:

```
i=100;
```

```
j=200;
```

Other forms of assignment operator exist that are obtaining various operators such as +, -, *, /, % etc with the = sign.

operator	Equation	Output
+=	i=i+1	i+=1
-=	i=i-10	i-=10
*=	i=i*11	i*=11
/=	i=i/12	i/=12
%=	i=i%5	i%=5

Conditional operator

Ternary (Conditional) operator:

☐☐ Conditional operator is an equivalent form of if –else loop.

☐☐ It is called as ternary operator because used on three operands.

Syntax:

true

(Condition) ? statement1 : statement2

false

This expression evaluates statement 1 or statement2 depends on the condition. If the condition is true statement1 gets executed other wise statement2 gets executed.

Ex:

```
main( )
{
int x;
clrscr( );
printf( " enter a no");
scanf("%d",&x);
(x%2==0) ? printf("even no") : printf("odd no");
getch();
```

The conditional expression can be used as shorthand for some if-else statements. It is a ternary operator. This operator consists of two symbols: the question mark (?) and the colon (:).

The general syntax of the conditional operator is:
Identifier = (test expression)? Expression1: Expression2 ;

This is an expression, not a statement, so it represents a value. The operator works by evaluating test expression. If it is true (non-zero), it evaluates and returns expression1. Otherwise, it evaluates and returns expression2.

The classic example of the ternary operator is to return the smaller of two variables. Every once in a while, the following form is just what you needed.

	Instead	of...
if (x < y)		
{		
min = x;		
}		
else		
{		
min = y;		
}		
You just say...		

min = (x < y) ? x : y;

Suppose that x and y are integer variables whose values are 100 and 4, respectively. After executing above statement, the value of min is 100.

Comma operator

A set of expression separated by comma is a valid constant in the C language. For example i and j are declared by the statements

```
int i, j;  
i=(j=10,j+20);
```

In the above declaration, right hand side consists of two expressions separated by comma. The first expression is j=10 and second is j+20. These expressions are evaluated from left to right. ie first the value 10 is assigned to j and then expression j+20 is evaluated so the final value of i will be 30.

Unary Operators

The classes of operator that act upon a single operand to produce a new value are known as Unary operator. The frequently used Unary operators in C are

1. Unary minus
2. Increment and decrement operator
3. Sizeof operator
4. Cast operator

Unary minus Operator

The operand of Unary minus operator must have arithmetic type, and the result is the -ve of its operand. The operand can be numerical constant, variable or expression. It is different from arithmetic subtraction operator (requires two operand).

Example:

$-10*(p+q)$

$-(p+q)$
 $x=-y$

Increment and decrement operators

C offers two unusual and unique operators ++ and -- called increment and decrement operator respectively. The use of these two operators will results in the value of the variable being incremented or decremented by unity. $i=i+1$ can be written as $i++$ and $j=j-1$ can be written as $j--$.

The operators can be place either before or after the operand. If the operator is placed before the variable ($++I$ or $--I$) it is known as pre incrementing or pre decrementing.

If the operator is placed after the variable ($I++$ or $I--$) it is known as post incrementing or post decrementing.

Pre incrementing or pre decrementing or post incrementing or post decrementing have different effect when used in expression. In the pre increment or pre decrement first the value of operand is incremented or decremented then it is assigned.

Example:

$x=100;$

$y=++x;$

After the execution the value of y will be 101 and the value of x will be 100.

In the post increment or post decrement first the value of operand is assigned to the concern variable and then the increment or decrement perform.

Example:

$x=100;$

$Y=x++;$

After the execution the value of y will be 100 and the value of x will be 101.

sizeof operator

The sizeof operator returns the size of its operand in bytes. The sizeof operator always precedes its operand. The operand may be an expression or it may be a cast.

Example:

Suppose i is an integer variable and c is a character type variable then

```
Printf("Size of integer : %d",sizeof(i));
Printf("Size of character : %c",sizeof(c));
Might generate the following output :
Size of integer : 2
Size of character : 1
```

Cast operator

Cast operator can be used to explicitly convert the value of an expression to a different data type. To do so, the name of the data type to which the conversion is to be made is enclosed in parenthesis and placed directly to the left of the expression whose value is to be converted. Its general format is (data type) expression

Example:

```
int i = (int)3.14;
float j = (float)10/2
```

Bitwise Operators

Bitwise operators operate on individual bits of integer (int and long) values. This is useful for writing low-level hardware or OS code where the ordinary abstractions of numbers, characters, pointers, and so on, are insufficient. Bit manipulation code tends to be less "portable". Code is "portable" if without any programmer intervention it compiles and runs correctly on different types of computers. The bit wise operations are commonly used with unsigned types. These operators perform bit wise logical operations on values. Both operands must be of the same type and width: the resultant value will also be this type and width.

Biwise operators are:

```
~ Bitwise Negation one's complement (unary)
& Bitwise And
| Bitwise Or
^ Bitwise Exclusive Or
>> Right Shift by right hand side (RHS) (divide by power of 2)
<< Left Shift by RHS (multiply by power of 2)
```

Definition of bitwise logical operators:

<u>Bit a</u>	<u>Bit b</u>	<u>a & b</u>	<u>a b</u>	<u>a ^ b</u>	<u>~a</u>
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Example:

Bitwise			Negation		
			a=00000011		
~a =11111100					
Bitwise			And		
a	=		00000011		
b	=		00010110		
a&b=00000010					
Bitwise			Or		
a	=		00000011		
b	=		00010110		
a b=00010111					
Bitwise			Exclusive		
a	=		00000011		
b	=		00010110		
a^b=00010101					
Right			Shift		
a	=	0000	0011	=	3
(a<<=1)	=		00000110	=	6
(a<<=2)	=		00011000	=	24
(a<<=3) = 11000000 = 192					
Left			Shift		
a=11000000			=192		
(a>>=1)	=		01100000	=	96
(a>>=2)	=		00011000	=	24
(a>>=3) = 0000 0011 = 3					

Comments

Comments are added to make a program more readable to you but the compiler must ignore the comments. Everything that is inside `/*` and `*/` is considered a comment and will be ignored by the compiler. You must not include comments within other comments, so something like this is not allowed: `[*]`.

`/* this is a /* comment */ inside a comment, which is wrong! */`

Comments are a way of inserting remarks and reminders into a program without affecting its content. Comments do not have a fixed place in a program: the compiler treats them as though they were white space or blank characters and they are consequently ignored. Programs can contain any number of comments without losing speed. This is because comments are stripped out of a source program by the compiler when it converts the source program into machine code.

Comments are marked out or delimited by the following pairs of characters:

```
/* ..... comment .....*/
```

Because a comment is skipped over as though it were a single space, it can be placed anywhere where spaces are valid characters, even in the middle of a statement, though this is not to be encouraged. You should try to minimize the use of comments in a program while trying to maximize the readability of the program. If there are too many comments you obscure your code and it is the code which is the main message in a program.

Float constants : 0.2 , 876.345, .345623 , 23.4E+8, 47.

Operators : a symbol, which indicates an operation to be performed.

Operators are used to manipulate data in program.

Delimiters : Language Pattern of c-language uses special kind of symbols

: (colon, used for labels) ; (semicolon terminates statement) ()

parameter list [] (array declaration and subscript), { } (block statement)

(hash for preprocessor directive) , (comma variable separator)

Conversion Specifiers

Code	Format
%a	Hexa decimal output in the form of 0xh.hhhhp+d(C99 only)
%s	String of characters (until null zero is reached)
%c	Character
%d	Decimal integer
%f	Floating-point numbers
%e	Exponential notation floating-point numbers
%g	Use the shorter of %f or %e
%u	Unsigned integer
%o	Octal integer
%x	Hexadecimal integer
%i	Signed decimal integer
%p	Display a pointer
%n	The associated argument must be a pointer to integer, This sepecifier causes the number of characters written in to be stored in that integer.
%hd	short integer
%ld	long integer
%lf	long double
%%	Prints a percent sign (%)

Precedence and associativity of operators

<u>Precdence Group</u> (Highest to Lowest)	<u>Operators</u>	<u>Associativity</u>
(param) subscript etc.,	() [] ->.	L → R
Unary operators	- + ! ~ ++ -- (type) * & sizeof	R → L
Multiplicative	* / %	L → R
Additive	+ -	L → R
Bitwise shift	<< >>	L → R
Relational	< <= > >=	L → R
Equality	= !=	L → R
Bitwise AND	&	L → R
Bitwise exclusive OR	^	L → R
Bitwise OR		L → R
Logical AND	&&	L → R
Logical OR		L → R
Conditional	?:	R → L
Assignment	= += -= *= /= %= &= ^=	R → L
Comma	,	L → R

Data types

z Integral Types

- ç **char** Stored as 8 bits. Unsigned 0 to 255.
Signed -128 to 127.
- ç **short int** Stored as 16 bits. Unsigned 0 to 65535.
Signed -32768 to 32767.
- ç **int** Same as either short or long int.
- ç **long int** Stored as 32 bits. Unsigned 0 to 4294967295.
Signed -2147483648 to 2147483647

z Floating Point Numbers

- ç Floating point numbers are rational numbers. Always signed numbers.
- ç **float** Approximate precision of 6 decimal digits .
 - z Typically stored in 4 bytes with 24 bits of signed mantissa and 8 bits of signed exponent.
- ç **double** Approximate precision of 14 decimal digits.
 - z Typically stored in 8 bytes with 56 bits of signed mantissa and 8 bits of signed exponent.
- ç One should check the file limits.h to what is implemented on a particular machine.

2 D Array

Many applications require that data be organized in more than one dimension. One common example is a matrix (a table), which is an array that consists of rows and columns.

Syntax:

Type arrayname[size of dimension 1][size of dimension 2];

Dimension 1 denotes rows

Dimension 2 denotes columns

E.g.

```
int table [3][3];
```

denotes a 2 dimensional array with 3 rows and 3 columns.

Declaration and Definition

Like one dimensional array, declaration of 2D array tells the compiler the name of the array, the type of its elements, and the size of each dimension.

E.g.1

```
Int table[4][4]={0,0,0,0,1,1,1,1,2,2,2,2,3,3,3,3};
```

E.g. 2

```
int table[4][4]={  
    {0,0,0,0},  
    {1,1,1,1},  
    {2,2,2,2},  
    {3,3,3,3}  
};
```

E.g.3

```
int table[][3]= {  
    {0,0,0},  
    {1,1,1}  
};
```

E.g.4

```
int table[2][3]={  
    {1,1},  
    {2}  
};
```

this will initialize first two elements of row one to 1, first element of row two to 2 and rest of the elements as 0.

E.g.5

```
Int arr[3][5]={ {0}, {0}, {0} };
```

This will initialize all the elements to zero.

Inputting values

```
int table[3][4];
```

```
for(i=0;i<3;i++)
```

```
{
```

```

for(j=0;j<4;j++)
{
scanf("%d",&table[i][j]);
}
}

```

Outputting values

```

for(i=0;i<3;i++)
{
for(j=0;j<4;j++)
{
printf("%d",table[i][j]);
}
}
}

```

Multidimensional array

C allows arrays of three or more dimensions.

Syntax:

Type nameof array[d1][d2][d3]....[dn];

E.g. int table [3][5][4];

```

int table[2][2][2]={
    {
        {1,1}
        {1,1}
    },
    {
        {2,2}
        {2,2}
    }
};

```